Clase File

Las primeras versiones de Java incluyen el paquete **java.io**, que **contiene casi todas las clases que podríamos necesitar para realizar operaciones de entrada y salida**.

La clase File es una **representación de nombres de rutas de ficheros y directorios, junto con los métodos necesarios para manipularlos**.

Un **objeto File NO es el fichero real,** por lo tanto no puede leer ni escribir en fichero.

Es un objeto que contiene **métodos que afectan a un archivo o directorio en particular y las funciones que los manipulan**.

Clase File

Constructor de File

File(String pathName) //Constructor

```
File f = new File("C:\\Users\\temp\\data.txt");
```

- pathName es una secuencia de nombres de directorio seguidos de un nombre de archivo.

Los nombres de los directorios están separados por un carácter especial que va a depender del sistema operativo.

¿Para qué sirve?

La clase File se utiliza para:

- Crear nuevos archivos y directorios.
- Eliminar archivos y directorios.
- Inspeccionar propiedades de archivos y directorios (como tamaño, permisos, etc.).
- Listar el contenido de directorios.
- Renombrar archivos y directorios.

Principales métodos de la clase File

- **exists()**: Comprueba si el archivo o directorio existe.
- **createNewFile()**: Crea un nuevo archivo vacío.
- **mkdir()**: Crea un nuevo directorio.
- **delete()**: Elimina el archivo o directorio.
- **list()**: Lista los nombres de los archivos y directorios en un directorio.
- **length()**: Devuelve el tamaño del archivo en bytes.
- **renameTo(File dest)**: Renombra el archivo o directorio.
- **isFile():** comprueba si el path del fichero se corresponde con un archivo
- **isDirectory():** comprueba si el path del fichero se corresponde con un directorio

Desventajas de la clase File

- **Manejo de errores:** Manejo deficiente de errores. No muestra detalles sobre los problemas encontrados,
- **No admite enlaces simbólicos** (accesos directos),
- **Compatibilidad**: Algunos métodos, como **rename()**, no funcionan de manera consistente en todas las plataformas,
- **No está optimizada para operaciones de E/S** que manejan grandes cantidades de datos, por lo tanto tiene un rendimiento limitado.
- **Debido a este rendimiento limitado, no es la mejor opción para manejar muchos archivos simultáneamente**,
- Debido a algunos de estos inconvenientes, Oracle desarrolló la API NIO mejorada.

Ejercicio

A partir de un fichero definido por la clase File, y de sus métodos, realiza las siguientes acciones:

1. Crea un nuevo archivo.
2. Crea un nuevo directorio.
3. Lista el contenido de un directorio.
4. Elimina un archivo
5. Renombra un archivo
6. Verificar si un archivo existe

Sentencia try-with-resources

- Sentencia try que declara uno o más recursos.
- **Recurso**: objeto que debe cerrarse después de que el programa termine con él.
- Se **garantiza que cada recurso se cierre al final de la instrucción.**
- Cualquier objeto que implemente la interfaz **Closeable**, que a su vez implementa de **AutoCloseable,** se puede utilizar **como recurso y no haría falta recurso.close()**
- Antes de Java SE 7, se usaba un **bloque finally para asegurarse de que un recurso se cerraba**, independientemente de si el try generaba excepción o no.

Ejemplo sin try-with-resource

```
public static void main(String[] args) {
        FileWriter fw =null;
        try {
            fw = new FileWriter("prueba.txt");
            fw.write("texto de prueba");
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try {
                if (fw != null) {
                    System.out.println("El fichero se cierra");
                    fw.close();
                }
            }catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
```

tenemos que crear el objeto FileWriter antes del try

en vez de hacerlo aquí

porque el finally se encuentra en otro scope y la variable fw no exisiría

Además, tendríamos que hacer **otras comprobaciones en el finally y también tendríamos que capturar excepción que lanza close()**, con lo que el programa quedaría bastante engorroso.

Ejemplo con try-with-resource

```
public static void main (String[] args) throws IOException {
    try(FileWriter fw = new FileWriter("prueba.txt");
        FileWriter fw2 = new FileWriter("prueba2.txt");) {
        //si quisiera poner un segundo writer quedaría así

        fw.write("texto de prueba");
    }
}
```

Podemos declarar varios recursos dentro de un mismo try.

Al final no hace falta cerrarlos, Ya se encarga la sentencia "try-with-resource"

- **Sin usar la sentencia try-with-resources:** si se lanzara una excepción al escribir el fichero, ésta se capturaría y se ejecutaría el finally a continuación. Si éste a su vez, lanzara también una excepción al llamar al método close, se reenviaría esta, NO la primera que se generó.
- **Mejora al usar try-with-resources:** se enviaría directamente la primera excepción que se generó. Lo cual tiene sentido, puesto que fue la causante del error.

Clase Path – java.nio.file

- Está **dentro del paquete java.nio.file.**
- Esta clase **proporciona una API más moderna y robusta para trabajar con archivos y directorios**..
- Además, al igual que la clase File, **Path también crea un objeto que se puede usar para ubicar un archivo en un sistema de archivos**.
- **Path puede realizar todas las operaciones que se pueden realizar con la clase File**.
- Muchos ficheros utilizan la notación "." para denotar el directorio actual y ".." para denotar el directorio del padre.

Path se encuentra en java.nio.file

Y se trabaja con las siguientes clases:

- **java.nio.file.Path**: Es una **interfaz que localiza un fichero o directorio mediante una ruta** dependiente del sistema.
- **java.nio.file.Files**: En combinación con Path, **realiza operaciones en ficheros o directorios**(escritura, lectura, …).
- **java.nio.file.FileSystem**: Proporciona una interfaz para el sistema de ficheros y una fábrica para crear Path y otros objetos que acceden al sistema de ficheros.
- Todos **los métodos que acceden al sistema de ficheros lanzan una excepción de tipo IOException**.

File vs Path

En Java, **Path y File son clases responsables de las operaciones de E/S de ficheros**. Realizan las mismas funciones pero pertenecen a diferentes paquetes.

Para construir un objeto:

- File lo hace a través del constructor: **File file = new File("ada.txt");**
- Path usa un método estático: **Path path = Paths.get("ada.txt");**

Escritura en fichero con “Path”

```
//Escritura con clase Path directamente
List<String> lista = new ArrayList<String>();
lista.add("Ejemplo de escritura utilizando la clase Path");
lista.add("Se utilizan métodos estáticos");
Path p1 = Paths.get( first: "./src/ficheros/fichPath.txt");
try {
    Files.write(p1,lista);
} catch (IOException e) {
    System.out.println("No se ha podido escribir en el fichero");
}
```

Escritura en fichero con “Path”

```
//Escritura con clase path utilizando el FileSystem
Path p = FileSystems.getDefault().getPath( first: "./src/ficheros/ficPathFS.txt");
try {
    BufferedWriter bw = Files.newBufferedWriter(p);
    bw.write( str: "hola en fichero con FileSystem");
    bw.close();
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

Lectura en fichero con “Path”

```
//Con las clases Path y Files
Path p1 = Paths.get( first: "./src/ficheros/fichPath.txt");
try {
    List<String> lista = Files.readAllLines(p1);
    for (String s:lista)
        System.out.println(s);
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

Lectura en fichero con “Path”

```
//Con la clase Path y FileSystem y Files
Path p2 = FileSystems.getDefault().getPath( first: "./src/ficheros/ficPathFS.txt");
try {
    BufferedReader br = Files.newBufferedReader(p2);
    System.out.println(br.readLine());
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

Ejercicio

Realizar las mismas acciones que vimos con la clase File, pero utilizando Path y Files y sus métodos correspondientes.

1. Crea un nuevo archivo.
2. Crea un nuevo directorio.
3. Lista el contenido de un directorio.
4. Elimina un archivo
5. Renombra un archivo
6. Verificar si un archivo existe